Análisis de Complejidad

.

Análisis de complejidad del Requerimiento 2: David Herrera

La complejidad del requerimiento 2 es:

$$\boxed{O(f)}$$

Abajo está el razonamiento para esto.

- c_1: asignaciones.
- c_2: `get_time()` y `delta_time()`.
- c_3: `digraph.get_vertex_information()`.
- c_4: `linear_map.key_set()`.
- c_5: acceso con `[]`.
- c_6: comparaciones $>$.
- c_7: `array_list.new_list()` y `array_list.add_last()`.

Donde

- f es la cantidad de amigos de la cuenta A, es decir, el tamaño de `informacion_A["FRIENDS"]`.

Justificación

- Antes del ciclo, se toma el tiempo inicial y se obtiene la información del usuario A. Estas operaciones son constantes.
- Luego se obtiene la lista de amigos de A con `linear_map.key_set()`. Esta operación depende de la cantidad de amigos de A, por eso cuesta $O(f)$.
- El ciclo `for` recorre todos los amigos de A. Por eso se ejecuta f veces.
- Dentro del ciclo se obtiene la información de cada amigo, se consulta su cantidad de seguidores y se compara con el mayor valor encontrado hasta el momento. Todo esto es constante.

- Después del ciclo se crea una lista con la información del usuario más popular. Esta parte es constante.

Entonces:

$$T(f) = f$$

Por lo tanto:

$$\boxed{O(f)}$$

```
def req_2(catalog, id_A):
    tiempo_inicial = get_time()

    informacion_A = digraph.get_vertex_information(catalog, id_A)
    amigos_A = linear_map.key_set(informacion_A["FRIENDS"])
    usuario_mas_popular = informacion_A
    mayor_cantidad_seguidores = informacion_A["FOLLOWERS"]

    for id_amigo in amigos_A["elements"]:

        informacion_amigo = digraph.get_vertex_information(catalog,
           id_amigo)
        seguidores_amigo = informacion_amigo["FOLLOWERS"]

        if seguidores_amigo > mayor_cantidad_seguidores:

            usuario_mas_popular = informacion_amigo
            mayor_cantidad_seguidores = seguidores_amigo

    informacion_resultado = array_list.new_list()

    array_list.add_last(informacion_resultado,usuario_mas_popular["
       USER_ID"])
    array_list.add_last(informacion_resultado,usuario_mas_popular["
       USER_NAME"])
    array_list.add_last(informacion_resultado,usuario_mas_popular["
       USER_TYPE"])

    tiempo_final = get_time()
    tiempo = delta_time(tiempo_inicial, tiempo_final)

    return tiempo, informacion_resultado, mayor_cantidad_seguidores
```

Análisis de complejidad del Requerimiento 5: David Herrera

La complejidad del requerimiento 5 es:

$$\boxed{O(V \log V + N(V + E))}$$

Abajo está el razonamiento para esto.

- c_1: asignaciones.
- c_2: `get_time()` y `delta_time()`.
- c_3: `digraph.vertices()`.
- c_4: `priority_queue.new_heap()`.
- c_5: `digraph.get_vertex_information()`.
- c_6: `array_list.new_list()`, `array_list.add_last()` y `array_list.get_element()`.
- c_7: `priority_queue.insert()`.
- c_8: `priority_queue.remove()`.
- c_9: `bfs.bfs()`.
- c_{10}: `bfs.has_path_to()`.
- c_{11}: `linear_map.new_map()`, `linear_map.get()` y `linear_map.put()`.

Donde

- V es la cantidad de usuarios del grafo, es decir, la cantidad de vértices.
- E es la cantidad de conexiones del grafo, es decir, la cantidad de arcos.
- N es el número de usuarios más populares que se quieren consultar, es decir, `numero_populares`.

Justificación

- Primero se toma el tiempo inicial. Esto es constante.
- Luego se obtiene la lista de usuarios del grafo con `digraph.vertices(catalog)`. Esta operación depende de la cantidad de vértices, por lo tanto cuesta $O(V)$.
- Después se crea una cola de prioridad, lo cual se toma como constante.
- El primer ciclo recorre todos los usuarios del grafo.
- Como hay V usuarios, este ciclo se ejecuta V veces.

- En cada iteración se obtiene la información del usuario, se crea una lista con su id, nombre y seguidores, y se inserta en la cola de prioridad.
- Como la cola de prioridad está implementada como heap, cada inserción cuesta $O(\log V)$.
- Por eso, insertar los V usuarios en la cola cuesta:

$$O(V \log V)$$

Luego se extraen los N usuarios más populares:

- El ciclo `while` se ejecuta máximo N veces.
- Cada extracción de la cola de prioridad cuesta $O(\log V)$.
- Entonces, esta parte cuesta:

$$O(N \log V)$$

Después se busca si alguno de los N usuarios populares puede ser raíz del árbol:

- Se recorren los N usuarios populares como posibles raíces.
- Para cada candidato se ejecuta un BFS.
- Un BFS sobre el grafo puede recorrer todos los vértices y todos los arcos.
- Por eso, cada BFS cuesta:

$$O(V + E)$$

Como en el peor caso se pueden probar los N candidatos, esta parte cuesta:

$$O(N(V + E))$$

Además, por cada candidato se revisa si puede llegar a los otros usuarios populares:

- Para cada candidato se recorren los N usuarios populares.
- La función `bfs.has_path_to()` se toma como $O(1)$ porque consulta la información ya guardada por el BFS.
- Entonces, esta verificación cuesta $O(N)$ por candidato.
- En el peor caso, para los N candidatos cuesta:

$$O(N^2)$$

Finalmente, si existe árbol, se reconstruyen los caminos desde la raíz hacia los demás usuarios populares:

- Se recorren los N usuarios populares.
- Para cada usuario se puede reconstruir un camino usando `edge_from`.
- En el peor caso, un camino puede tener hasta V vértices.
- Entonces, esta parte puede costar $O(NV)$.
- Como $O(NV)$ también queda dentro de $O(N(V+E))$, no cambia la complejidad final.

Entonces:

$$\boxed{O(V \log V + N(V+E))}$$

```
def req_5(catalog, numero_populares):
    tiempo_inicial = get_time()

    usuarios = digraph.vertices(catalog)
    cola_popularidad = priority_queue.new_heap(is_min_pq=False)

    for id_usuario in usuarios["elements"]:
        usuario_info = digraph.get_vertex_information(catalog,
           id_usuario)

        informacion_usuario = array_list.new_list()
        array_list.add_last(informacion_usuario,usuario_info["
           USER_ID"])
        array_list.add_last(informacion_usuario,usuario_info["
           USER_NAME"])
        array_list.add_last(informacion_usuario,usuario_info["
           FOLLOWERS"])

        priority_queue.insert(
            cola_popularidad,
            usuario_info["FOLLOWERS"],
            informacion_usuario
        )

    usuarios_mas_populares = array_list.new_list()
    contador = 0

    while contador < numero_populares and not priority_queue.
       is_empty(cola_popularidad):
        usuario_popular = priority_queue.remove(cola_popularidad)
        array_list.add_last(usuarios_mas_populares,usuario_popular)

        contador += 1

    existe_arbol = False
```

```
    raiz_arbol = None
    recorrido_raiz = None

    for candidato_raiz in usuarios_mas_populares["elements"]:

        if not existe_arbol:
            id_candidato = array_list.get_element(candidato_raiz,0)
            recorrido_candidato = bfs.bfs(catalog,id_candidato)

            conecta_a_todos = True

            for usuario_objetivo in usuarios_mas_populares["elements
               "]:
                id_objetivo = array_list.get_element(
                   usuario_objetivo,0)

                if conecta_a_todos and not bfs.has_path_to(
                   id_objetivo,recorrido_candidato):
                    conecta_a_todos = False

            if conecta_a_todos:
                existe_arbol = True
                raiz_arbol = candidato_raiz
                recorrido_raiz = recorrido_candidato
```

Análisis de complejidad del Requerimiento 3: David Martinez

La complejidad del requerimiento 3 es:

$$\boxed{O(a)}$$

Abajo está el razonamiento para esto.

- c_1: asignaciones.
- c_2: `get_time()` y `delta_time()`.
- c_3: `digraph.get_vertex_information()`.
- c_4: `linear_map.key_set()`.
- c_5: `linear_map.get()`.
- c_6: acceso con `[]`.
- c_7: comparación `is not None`.
- c_8: `array_list.new_list()` y `array_list.add_last()`.

Donde

- a es la cantidad de amigos de la cuenta A.
- b es la cantidad de amigos de la cuenta B.
- c es la cantidad de amigos en común entre A y B, donde $c \leq a$.

Justificación

- Antes del ciclo principal, se obtiene la información de los usuarios A y B. Esto es constante.
- Luego se obtiene la lista de amigos de A usando `linear_map.key_set()`. Esta operación depende de la cantidad de amigos de A, por eso cuesta $O(a)$.
- También se obtiene el mapa de amigos de B. Esto se toma como constante, porque solo se accede a la estructura ya creada.
- El ciclo `for` recorre todos los amigos de A, por lo tanto se ejecuta a veces.
- En cada iteración se consulta si el amigo de A también está en el mapa de amigos de $Busando$`linear_map.get()`.
- Como `linear_map` es una tabla hash, esta búsqueda se considera $O(1)$ en promedio.

- Si el amigo está en común, se obtiene su información, se crea una lista con su id, nombre y tipo de usuario, y se agrega a la lista de respuesta. Estas operaciones son constantes.
- Como la cantidad de amigos en común c nunca puede ser mayor que a, esta parte no cambia el orden final de la función.

Entonces: Como $c \leq a$, se puede simplificar a:

$$\boxed{O(a)}$$

```
def req_3(catalog, id_A, id_B):
    tiempo_inicial = get_time()

    id_A_info = digraph.get_vertex_information(catalog, id_A)
    id_B_info = digraph.get_vertex_information(catalog, id_B)

    id_A_amigos_lista = linear_map.key_set(id_A_info["FRIENDS"])
    id_B_amigos_mapa = id_B_info["FRIENDS"]

    amigos_comun = array_list.new_list()

    for amigoA in id_A_amigos_lista["elements"]:
        if linear_map.get(id_B_amigos_mapa, amigoA) is not None:
            amigo_compartido = digraph.get_vertex_information(
               catalog, amigoA)

            informacion_amigo_compartido = array_list.new_list()
            array_list.add_last(informacion_amigo_compartido,
               amigo_compartido["USER_ID"])
            array_list.add_last(informacion_amigo_compartido,
               amigo_compartido["USER_NAME"])
            array_list.add_last(informacion_amigo_compartido,
               amigo_compartido["USER_TYPE"])

            array_list.add_last(amigos_comun,
               informacion_amigo_compartido)

    tiempo_final = get_time()
    tiempo = delta_time(tiempo_inicial, tiempo_final)

    return tiempo, amigos_comun
```

Requerimiento 6

las primeras lineas antes del for son constantes a excepcion de la linea de bfs $V + E$ (asumimos el numero de hobbies es constante) y keys, la cual obtiene todas llaves del mapa y por lo tanto es aproximadamente $2V$, ya que en el peor caso el mapa es casi el doble de los vertices (un caso donde un id es amigo de todos). Ahora, debido a que asumimos que hobbies es constante, el for solo lo recorremos V veces. Por lo tanto, al complejidad es $O(V + E)$

In [ ]:
```
def req_6(catalog, id, hobbies):
    """
    Retorna el resultado del requerimiento 6

    Se desea obtener una sub-red que conecte a los amigos de los amigos de u
    intereses (hobbies) similares. En este caso la amistad también se consid
    transitividad, es decir, si A es amigo de B y B es amigo de C, A y C se
    implícitos (sin necesidad de tener relación directa). A y B son amigos e
    profundidad, B y C son amigos explícitos a 1 arco de profundidad, A y C
    a 2 arcos de profundidad.

    Los parámetros de entrada de este requisito son:
        - Cuenta (Id de usuario) de donde partirá la búsqueda.
        - El (los) hobbie(s) de interés para la búsqueda: array_list

    La respuesta esperada debe contener la siguiente información:
        - El tiempo que se demora el algoritmo en encontrar la solución.
        - La cantidad de amigos explícitos e implícitos de la cuenta A que s
        - La siguiente información de cada uno de los amigos encontrados:
            o Nivel de profundidad (número de arcos de conexión a la cuenta
            o El(los) hobbie(s) en común con la cuenta A.
    """
    # TODO: Modificar el requerimiento 6
    tiempo_i = get_time()
    id_info = digraph.get_vertex_information(catalog, id)

    id_hobbies = id_info["HOBBIES"]


    # Crear la sub-red filtrada
    visited_map = bfs_amigos(catalog, id, hobbies)

    amigos_explicitos = 0
    amigos_implicitos = 0

    # Array list donode cada elemento tiene [id, profundidad, [hobbies en co
    respuesta = array_list.new_list()

    keys = linear_map.key_set(visited_map)

    for key in keys["elements"]:
```

```
        hobbies_key = digraph.get_vertex_information(catalog, key)["HOBBIES"

        hobbies_comun = array_list.new_list()

        for hobby in hobbies_key["elements"]:
            if hobby in id_hobbies["elements"]:
                array_list.add_last(hobbies_comun, hobby)

        profundidad = linear_map.get(visited_map, key)["distance"]

        if profundidad == 1:
            amigos_explicitos += 1
        elif profundidad == 2:
            amigos_implicitos += 1

        amigo_encontrado = array_list.new_list()

        array_list.add_last(amigo_encontrado, key)
        array_list.add_last(amigo_encontrado, profundidad)
        array_list.add_last(amigo_encontrado, hobbies_comun)

        array_list.add_last(respuesta, amigo_encontrado)

    tiempo_f = get_time()
    tiempo = delta_time(tiempo_i, tiempo_f)

    return tiempo, respuesta, amigos_explicitos, amigos_implicitos


def bfs_amigos(my_graph, source, hobbies):
    """
    Inicia un recorrido Breath First Seacrh (BFS) sobre el grafo a partir de

    Parameters:

            my_graph (digraph) – El grafo a recorrer

            source (any) – Llave del vertice de inicio del recorrido.

    Returns: Mapa con la información de los vertices conectados a source (pa
    Return type: map
    """
    visited_map = linear_map.new_map(digraph.size(my_graph), 0.5)

    if digraph.get_vertex(my_graph, source) is None:
        raise Exception("Vertice no existe")

    linear_map.put(visited_map, source, {"marked": True, "edge_from": None,


    return bfs_vertex_amigos(my_graph, source, visited_map, hobbies)

def bfs_vertex_amigos(my_graph, source, visited_map, hobbies):
    """
    Función auxiliar para calcular un recorrido BFS usada por la función bfs
```

```
    Parameters:

            my_graph (digraph) – El grafo a recorrer

            source (any) – Vertice de inicio del recorrido.

            visited_map (map) – Mapa para almacenar el recorrido

    Returns: El mapa actualizado con los vertices alcanzables
    Return type: map
    """
    cola = queue.new_queue()

    queue.enqueue(cola, source)
    distancia = 0

    while not queue.is_empty(cola) and distancia < 3:
        vertice = queue.dequeue(cola)

        amigos = linear_map.key_set(digraph.get_vertex_information(my_graph,

        for i in amigos["elements"]:
            hobbies_i = digraph.get_vertex_information(my_graph, i)["HOBBIES
            if linear_map.get(visited_map, i) is None:
                if (linear_map.get(visited_map, vertice)["distance"] + 1 ==
                    linear_map.put(visited_map, i, {"marked": True, "edge_fr
                    queue.enqueue(cola, i)
                else:
                    # Mirar si todos los hobbies se cumplen
                    cumpleHobbies = True

                    indice = 0
                    while indice < array_list.size(hobbies) and cumpleHobbie
                        if array_list.get_element(hobbies, indice) not in ho
                            cumpleHobbies = False

                        indice += 1

                    if cumpleHobbies:
                        linear_map.put(visited_map, i, {"marked": True, "edg
                        queue.enqueue(cola, i)

        if not queue.is_empty(cola):
            distancia = linear_map.get(visited_map, queue.peek(cola))["dista

    linear_map.remove(visited_map, source)

    return visited_map
```

ANÁLISIS DEL RETO

Estudiante 1 Edgar Andres Barahona Pinilla, código 202416331, email e.barahona@uniandes.edu.co

Requerimiento <<4>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Vamos a insertar a cada amigo del usuario en una cola de prioridad (como prioridad la cantidad de usuarios que siguen los amigos) y extraemos los N amigos

Entrada	Grafo de la red social, ID del usuario y número de amigos a consultar.
Salidas	Tiempo de ejecución y lista con los **N** amigos que siguen a más usuarios.
Implementado (Sí/No)	Si

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Obtener la lista de amigos del usuario	O(F)
Crear la cola de prioridad	O(1)
Recorrer amigos insertando cada uno en la cola prioridad	O(F log F)
Extraer los N amigos con mayor prioridad	O(N)
TOTAL	***O(F log F + N)***

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Requerimiento <<1>>

Descripción

Usamos BFS para encontrar el camino de menor extension entre dos usuarios del tipo basic

Entrada	Grafo de la red social, ID del usuario y número de amigos a consultar.
Salidas	Tiempo de ejecución y lista con los **N** amigos que siguen a más usuarios.

Implementado (Sí/No)	Si

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Obtener la información usuario A y b	O(1)
Verificar que los dos sean basic	O(1)
Hacer BFS sobre el grafo	O(V+E)
Path_to	O(P)
Recorrer (path_to) obteniendo la info de cada usuario	O(P)
TOTAL	***O(V+E)***

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.